
University of Pisa

Department of Computer Science
Bachelor's Degree

Bachelor's thesis

# **here title**
# **here title**

Supervisors:

**Prof. Anna Bernasconi**
University of Pisa

**Dott. Giorgio dell'Immagine**
**non lo so**

Candidate:

**Sofia Scalzo**
University of Pisa

Academic year 2025/2026

# Abstract

# Contents

# Chapter 1

# Mathematical and Cryptographic Background

This chapter introduces the notation and mathematical foundations used throughout the thesis.

## 1.1 Finite Fields

**Definition 1.1** (Prime Field)**.** Let $p$ be a prime number. The *prime field* $\mathbb{F}_p$ is the set $\{0, 1, \ldots, p-1\}$ equipped with addition and multiplication modulo $p$.

The multiplicative group $\mathbb{F}_p^* = \mathbb{F}_p \setminus \{0\}$ is cyclic of order $p-1$. An element $g \in \mathbb{F}_p^*$ is called a *generator* if $\{g^0, g^1, \ldots, g^{p-2}\} = \mathbb{F}_p^*$. For any $a \in \mathbb{F}_p^*$, the multiplicative inverse can be computed via Fermat's little theorem:

$$a^{-1} \equiv a^{p-2} \pmod{p}. \tag{1.1}$$

### 1.1.1 The BabyBear Field

The implementation described in this thesis operates over the BabyBear prime field, defined by

$$p = 2^{31} - 2^{27} + 1 = 2\,013\,265\,921. \tag{1.2}$$

Since $p < 2^{31}$, every field element fits in a 32-bit unsigned integer, and the product of two elements fits in a 64-bit integer.

The multiplicative group $\mathbb{F}_p^*$ has order $p-1 = 2^{27} \cdot 15$. The factor $2^{27}$, the *two-adicity* of the field, means that $\mathbb{F}_p^*$ contains a cyclic subgroup of order $2^{27}$, which is essential for the Number Theoretic Transform.

### 1.1.2 Roots of Unity

**Definition 1.2** (Primitive Root of Unity)**.** An element $\omega \in \mathbb{F}_p$ is a *primitive $N$-th root of unity* if $\omega^N = 1$ and $\omega^k \neq 1$ for all $0 < k < N$.

Given a primitive $N$-th root of unity $\omega$, the set $H = \{1, \omega, \omega^2, \ldots, \omega^{N-1}\}$ is a multiplicative subgroup of $\mathbb{F}_p^*$ of order $N$. Because BabyBear has two-adicity 27, subgroups

of order $2^k$ for any $k \leq 27$ exist and are generated from a fixed multiplicative generator $g$ by taking $g^{(p-1)/2^k}$.

These subgroups serve as evaluation domains for the polynomials in the STARK protocol: the execution trace is evaluated on a subgroup $H$ of order $2^h$, and FRI operates on a larger domain.

### 1.1.3 Field Extensions

**Definition 1.3** (Field Extension)**.** A *field extension* $\mathbb{F}_{p^k}$ of degree $k$ over $\mathbb{F}_p$ is the quotient ring $\mathbb{F}_p[X]/(f(X))$, where $f(X) \in \mathbb{F}_p[X]$ is an irreducible polynomial of degree $k$. Elements are polynomials of degree less than $k$ with coefficients in $\mathbb{F}_p$.

In this work we use the *quartic extension* $\mathbb{F}_{p^4}$, constructed via the irreducible polynomial $X^4 - 11$. The constant 11 is chosen because it is not a fourth power in $\mathbb{F}_p$, which guarantees that $X^4 - 11$ is irreducible. An element of $\mathbb{F}_{p^4}$ is a tuple $(c_0, c_1, c_2, c_3) \in \mathbb{F}_p^4$ representing $c_0 + c_1 X + c_2 X^2 + c_3 X^3$, with the reduction rule $X^4 = 11$.

## 1.2 Polynomials

A univariate polynomial over $\mathbb{F}_p$ of degree at most $d$ is $f(X) = \sum_{i=0}^{d} c_i X^i$ with $c_i \in \mathbb{F}_p$. It can be represented by its *coefficient vector* $(c_0, \ldots, c_d)$ or by its *evaluation vector* on a domain $D$. Converting between the two representations, and evaluating a polynomial at a point, are the fundamental operations of the prover.

### 1.2.1 The Number Theoretic Transform

The conversion between coefficient and evaluation representations is performed using the Number Theoretic Transform (NTT), the finite-field analogue of the Fast Fourier Transform. When the evaluation domain is a multiplicative subgroup $D = \{1, \omega, \ldots, \omega^{N-1}\}$ with $N$ a power of 2, the NTT computes all evaluations $f(\omega^i)$ in $O(N \log N)$ operations. The algorithm exploits the decomposition

$$f(X) = f_{\text{even}}(X^2) + X \cdot f_{\text{odd}}(X^2),$$

recursing on the even- and odd-indexed coefficients over the squared subgroup, which has half the size. The inverse transform runs the same routine with $\omega^{-1}$ and divides by $N$.

### 1.2.2 Low-Degree Extension

The *low-degree extension* (LDE) of a polynomial is its evaluation on a domain larger than its coefficient count. Given the $n$ coefficients of $f$, the LDE with blowup factor $b$ evaluates $f$ on a coset of a subgroup of size $b \cdot n$. The resulting evaluation vector is the Reed-Solomon codeword that the prover commits to. In our implementation the low-degree extension is computed by padding the coefficients with zeros, applying a coset shift, and running the NTT on the enlarged domain [Sta21, Section 3.4].

### 1.2.3 Vanishing Polynomials

**Definition 1.4** (Vanishing Polynomial [Sta21, Section 5.1])**.** Given a set $S \subseteq \mathbb{F}$, the *vanishing polynomial* of $S$ is $Z_S(X) := \prod_{s \in S}(X - s)$. When $S$ is a multiplicative subgroup $H = \{1, \omega, \ldots, \omega^{N-1}\}$, this simplifies to $Z_H(X) = X^N - 1$.

$Z_H$ vanishes on every element of $H$ and nowhere else. If a constraint polynomial $c(X)$ must be zero on $H$, then $Z_H(X)$ divides $c(X)$, and the quotient $q(X) = c(X)/Z_H(X)$ is a polynomial. The verifier checks that $q$ has low degree, which certifies that $c$ vanishes on $H$ [Sta21]. This mechanism is the basis of the STARK construction.

## 1.3 Coding Theory

Reed-Solomon codes, and distance from them, are the objects the FRI protocol operates on. We recall the general definitions that the soundness analysis of the FRI protocol relies on.

**Definition 1.5** (Error-Correcting Code [Arn+24, Definition 3.1])**.** An *error-correcting code* of length $n$ over an alphabet $\Sigma$ is a subset $C \subseteq \Sigma^n$. The code is a *linear code* if $\Sigma = \mathbb{F}$ is a field and $C$ is a linear subspace of $\mathbb{F}^n$. The *rate* of a linear code is $\rho := \dim(C)/n$.

The elements of $C$ are its *codewords*, and the *minimum distance* of a code is the smallest relative Hamming distance (definition 1.8) between two distinct codewords. When a word lies far from every codeword, several codewords may still lie within a given distance of it; the following notion quantifies this.

**Definition 1.6** (List Decoding [Arn+24, Definition 3.2])**.** For a code $C \subseteq \Sigma^n$, a word $f \in \Sigma^n$, and a proximity parameter $\delta \in [0, 1]$, the *list* of $C$ at $f$ within radius $\delta$ is

$$\Lambda(C, f, \delta) := \{u \in C : \Delta(f, u) \leq \delta\}.$$

The code is $(\delta, \ell)$-*list decodable* if $|\Lambda(C, f, \delta)| \leq \ell$ for every $f \in \Sigma^n$. A radius $\delta$ is *within unique decoding* if the code is $(\delta, 1)$-list decodable.

For a code of minimum distance $\delta_C$, any radius $\delta \leq \delta_C/2$ is within unique decoding. Beyond it, the list size for Reed-Solomon codes is controlled by the Johnson bound.

**Theorem 1.1** (Johnson Bound [Arn+24, Theorem 4.3])**.** *The Reed-Solomon code of rate $\rho$ is $\left(1 - \sqrt{\rho} - \eta,\ \frac{1}{2\eta\sqrt{\rho}}\right)$-list decodable for every $\eta \in (0, 1 - \sqrt{\rho})$.*

The two thresholds — the *unique decoding* radius $\frac{1-\rho}{2}$ and the *Johnson* radius $1-\sqrt{\rho}$ — are exactly where the soundness guarantees of FRI change form (chapter 2).

## 1.4 Reed-Solomon Codes

**Definition 1.7** (Reed-Solomon Code [BS+18, Section 2.1])**.** For a finite field $\mathbb{F}$, a subset $S \subseteq \mathbb{F}$, and a rate parameter $\rho \in (0, 1)$, the *Reed-Solomon code* $\mathrm{RS}[\mathbb{F}, S, \rho]$ is the set of functions $f : S \to \mathbb{F}$ that are evaluations of polynomials of degree less than $\rho|S|$:

$$\mathrm{RS}[\mathbb{F}, S, \rho] = \{(f(x))_{x \in S} : f \in \mathbb{F}[X],\ \deg(f) < \rho|S|\}.$$

Each codeword is the evaluation of a polynomial of degree less than $\rho|S|$ on all points of $S$. The code has minimum distance $|S| - \rho|S| + 1$, so two distinct codewords differ on a large fraction of positions.

The parameter $\rho$ is the *rate*, and $1/\rho$ is the *blowup factor*. In our implementation the blowup factor is 2 ($\rho = 1/2$): the codeword produced by the low-degree extension is twice the length of the coefficient vector.

### 1.4.1 Relative Hamming Distance

**Definition 1.8** (Relative Hamming Distance)**.** The *relative Hamming distance* between two vectors $u, v \in \mathbb{F}^n$ is

$$\Delta(u, v) = \frac{|\{i : u_i \neq v_i\}|}{n}.$$

The *distance* of a vector $u$ from a code $C$ is $\Delta(u, C) = \min_{v \in C} \Delta(u, v)$. A vector $u$ is $\delta$-*close* to $C$ if $\Delta(u, C) \leq \delta$, and $\delta$-*far* if $\Delta(u, C) > \delta$.

The FRI protocol tests proximity to a Reed-Solomon codeword. As stated in [BS+18, Section 2.1], the verifier's task is to distinguish between the case that $f \in \mathrm{RS}[\mathbb{F}, S, \rho]$ and the case that $f$ is far from $\mathrm{RS}[\mathbb{F}, S, \rho]$ in relative Hamming distance.

## 1.5 Cryptographic Hash Functions

**Definition 1.9** (Collision-Resistant Hash Function)**.** A *collision-resistant hash function* is a function $H : \{0,1\}^* \to \{0,1\}^\lambda$ such that no efficient algorithm can find distinct inputs $x \neq x'$ with $H(x) = H(x')$, except with negligible probability.

Hash functions serve two roles in STARK systems: constructing Merkle trees for committing to polynomial evaluations, and deriving the verifier's challenges via the Fiat-Shamir heuristic. Our implementation uses two different hash functions for these two roles: SHA-256 for the Merkle commitments and Keccak-$f$[1600] for the challenger.

The distinction matters for the Fiat-Shamir transformation. Chiesa and Orrù [CO25] prove that a *duplex sponge* construction (such as Keccak/SHA-3) is suitable for Fiat-Shamir, while Merkle-Damgård constructions (such as SHA-256) lack a comparable security proof in this setting. For this reason we use a Keccak-based sponge for the challenger, while retaining SHA-256 for the Merkle trees, where only collision resistance is required.

## 1.6 Merkle Trees

**Definition 1.10** (Merkle Tree)**.** A *Merkle tree* over a sequence $(v_0, v_1, \ldots, v_{N-1})$ with $N = 2^k$ is a binary tree where each leaf stores the hash of one element, $\ell_i = H(v_i)$; each internal node stores the hash of the concatenation of its two children, $H(n_{\text{left}} \| n_{\text{right}})$; and the root $r$ is a single digest that binds the entire sequence.

To prove that $v_i$ is the value at position $i$, the prover provides an *authentication path*: the $\log_2(N)$ sibling hashes along the path from the leaf to the root. The verifier recomputes the root and accepts if it matches $r$. Verification requires $O(\log N)$ hash evaluations.

In the BCS transformation [BSCS16], Merkle trees commit to the oracle messages of the IOP: the prover evaluates a polynomial on the FRI domain, builds a Merkle tree over the evaluations, and sends the root to the verifier.

## 1.7 Proof Systems

We now establish the formal framework of proof systems, following Thaler [Tha22].

### 1.7.1 Interactive Proofs

**Definition 1.11** (Interactive Proof [Tha22, Definition 3.1])**.** An *interactive proof system* for a function $f$ is a pair $(\mathcal{P}, \mathcal{V})$ where $\mathcal{P}$ is a computationally unbounded prover and $\mathcal{V}$ is a probabilistic polynomial-time verifier. The protocol proceeds in rounds, and must satisfy:

- **Completeness.** For every input $x$, $\Pr[\mathcal{V} \text{ accepts}] \geq 1 - \delta_c$.
- **Soundness.** For every input $x$ and every cheating prover $\mathcal{P}^*$ that sends a value $y \neq f(x)$, $\Pr[\mathcal{V} \text{ accepts}] \leq \delta_s$.

### 1.7.2 Interactive Arguments

Relaxing the soundness of an interactive proof (definition 1.11) to hold only against efficient provers yields a weaker but more flexible notion.

**Definition 1.12** (Interactive Argument [Tha22, Section 3.2])**.** An *interactive argument* for a language $\mathcal{L}$ is an interactive proof in which the soundness condition is only required to hold against prover strategies that run in probabilistic polynomial time.

Both models share the same structure. In a *public-coin* protocol, every verifier message is a uniformly random challenge, independent of the prover's messages, and the verifier's decision is a deterministic function of the *transcript* — the ordered sequence of messages and challenges. These are exactly the protocols to which the Fiat-Shamir transformation (section 1.7.3) applies. An *argument of knowledge* further requires an efficient extractor $\mathcal{E}$ that recovers a valid witness from any convincing prover.

STARKs are *arguments*, not *proofs* in the formal sense: their soundness relies on the collision resistance of a hash function, which a computationally unbounded prover could break. The interactive oracle proofs of section 1.8 specialise this model by providing the prover's messages as oracles.

### 1.7.3 The Fiat-Shamir Transformation

A *non-interactive* proof consists of a single message from the prover to the verifier. The Fiat-Shamir transformation removes interaction from a public-coin protocol by replacing the verifier's random challenges with the output of a hash function applied to the transcript so far. Under the random oracle model, this preserves soundness. Chiesa and Orrù [CO25] give a formal treatment using a duplex sponge, which resolves the ambiguity between writing to and reading from the transcript by using the sponge's native *absorb* and *squeeze* operations. Our challenger implements this construction over Keccak-$f$[1600].

## 1.8 Interactive Oracle Proofs

**Definition 1.13** (Interactive Oracle Proof [BSCS16, Definition 2.2])**.** An *Interactive Oracle Proof* (IOP) for a language $\mathcal{L}$ is an interactive protocol between a prover $\mathcal{P}$ and a verifier $\mathcal{V}$ that proceeds in rounds. In each round:

1. the prover sends an *oracle message* $\pi_i$ (a function that the verifier may query at chosen positions);
2. the verifier sends a random challenge $r_i$.

After $k$ rounds, the verifier makes a bounded number of queries to $\pi_1, \ldots, \pi_k$ and outputs accept or reject.

The key difference from a standard interactive proof is that the verifier has *oracle access* to the prover's messages: it reads only a small number of positions rather than the full message. This is what enables succinct verification.

An IOP of *proximity* (IOPP) is an IOP in which the verifier tests whether the prover's initial oracle is close to a codeword of a specified code, rather than testing membership in a language [BS+18].

## 1.9 The BCS Transformation

The BCS transformation [BSCS16] compiles a public-coin IOP into a non-interactive argument in the random oracle model:

1. **Commitment.** Each oracle message is committed using a Merkle tree; the prover sends the root.
2. **Challenge derivation.** Each challenge is derived from the transcript via Fiat-Shamir.
3. **Query answering.** The prover opens the requested positions together with their Merkle authentication paths.
4. **Verification.** The verifier checks the paths, recomputes the challenges, and runs the IOP decision procedure.

This is precisely the structure of our FRI commit phase: each folding layer is committed with a Merkle tree, its root is absorbed into the challenger, and the next folding challenge is squeezed from it. The query and verification steps are performed when the verifier opens the Merkle paths and checks folding consistency.

## 1.10 Polynomial Commitment Schemes

**Definition 1.14** (Polynomial Commitment Scheme)**.** A *Polynomial Commitment Scheme* (PCS) consists of four algorithms:

- $\mathsf{Setup}(\lambda, d) \to \mathsf{pp}$: generates public parameters for polynomials of degree at most $d$.
- $\mathsf{Commit}(\mathsf{pp}, f) \to C$: produces a commitment $C$ to the polynomial $f$.
- $\mathsf{Open}(\mathsf{pp}, f, z) \to (v, \pi)$: produces a proof $\pi$ that $f(z) = v$.
- $\mathsf{Verify}(\mathsf{pp}, C, z, v, \pi) \to \{0, 1\}$: checks the evaluation proof.

The scheme must satisfy correctness and binding.

The STARK system implemented in this thesis uses a *FRI-based PCS* [BS+18]: a polynomial is committed via a Merkle tree over its low-degree extension, and evaluation proofs are constructed through the FRI low-degree testing protocol. This approach requires no trusted setup (transparent) and relies only on hash functions (post-quantum secure). The FRI protocol is described in detail in Chapter 2.

# Chapter 2

# The FRI Protocol

This chapter presents the Fast Reed-Solomon Interactive Oracle Proof of Proximity, the protocol at the core of the system implemented in this thesis. We begin by stating the problem FRI solves, then describe the algebraic structure of the evaluation domain, and finally the folding operation that drives the protocol.

## 2.1 Overview and Problem Statement

Recall from definition 1.7 that, for a subset $S \subseteq \mathbb{F}$ and a rate parameter $\rho \in (0, 1)$, the Reed-Solomon code $\mathrm{RS}[\mathbb{F}, S, \rho]$ consists of the evaluations on $S$ of polynomials of degree less than $\rho|S|$. The problem FRI addresses is the following.

**Definition 2.1** (Rate Proximity Testing (RPT) [BS+18, Section 2.1])**.** A verifier is given oracle access to a function $f : S \to \mathbb{F}$, together with auxiliary information supplied by a prover. The verifier must distinguish, with high probability and using only a small number of queries, between the case that $f \in \mathrm{RS}[\mathbb{F}, S, \rho]$ and the case that $f$ is far from every member of $\mathrm{RS}[\mathbb{F}, S, \rho]$ in relative Hamming distance (definition 1.8).

RPT is a special case of the low-degree testing problem, and Ben-Sasson et al. identify it as a major bottleneck for transparent proof systems [BS+18, Section 2.1].

A naive solution is instructive. The verifier could ask the prover to open $f$ at $\rho|S|$ positions, interpolate the unique polynomial of degree less than $\rho|S|$ through them, and check that it agrees with $f$ elsewhere. This fails on two counts: the number of openings is proportional to the degree bound, which can reach $2^{20}$ or more in practical instances, and every opening carries a Merkle authentication path of $\log |S|$ hash values (section 1.6), so the proof size becomes prohibitive.

FRI takes a different route. Instead of testing the degree directly, it *reduces* it: the prover repeatedly halves the degree of the committed polynomial using randomness supplied by the verifier, until the degree is small enough to send the remaining polynomial directly. Each reduction step is committed, and the verifier spot-checks the consistency of the steps at a small number of random positions. If the original function is far from the code, then with high probability at least one reduction is inconsistent, and the verifier detects it.

This yields a protocol with prover arithmetic complexity strictly linear in $|S|$ and verifier arithmetic complexity strictly logarithmic [BS+18, Section 2.1]. In the ter-

minology of section 1.8, FRI is an interactive oracle proof of proximity (IOPP) for Reed-Solomon codes: the prover's messages are committed codewords which the verifier queries at a bounded number of positions, rather than reading in full.

*Remark* 2.1. FRI establishes *proximity*, not membership. A successful verification does not certify that the committed vector is exactly a Reed-Solomon codeword, only that it is close to one in relative Hamming distance. This distinction is central to the soundness analysis of the FRI protocol.

## 2.2 The Evaluation Domain

The folding step of FRI relies on specific algebraic structure in the evaluation domain. We state the requirements first, then give the construction used in our implementation.

### 2.2.1 Requirements

The reduction from a polynomial $f$ to a polynomial of half the degree is carried out by pairing the evaluations of $f$ at $x$ and at $-x$. For this to be possible at every position, the domain must be closed under negation. Moreover, the folded polynomial is committed not on $D$ but on the squared domain $D^{(2)} := \{x^2 : x \in D\}$, and since the folding step is applied iteratively, $D^{(2)}$ must be closed under negation as well, and so on for every subsequent layer [Sta21, Section 3.9.1].

We therefore require a chain of domains

$$D = D_0, \quad D_{i+1} = D_i^{(2)}, \quad i = 0, 1, \ldots$$

such that every $D_i$ is closed under negation and $|D_{i+1}| = |D_i|/2$. These requirements are met by taking $D$ to be a coset of a multiplicative subgroup of order $2^k$ [Sta21, Section 3.9.1].

A coset is used rather than the subgroup itself so that the evaluation domain is disjoint from the trace domain on which the polynomial being committed is defined [Sta21, Section 3.4]. Disjointness is required in the full STARK protocol, where constraint polynomials are expressed as rational functions whose denominators vanish on the trace domain; evaluating them on a disjoint domain keeps those denominators nonzero.

### 2.2.2 Construction

We build the domain from the roots of unity of section 1.1.2. Let $n$ be the number of coefficients of the polynomial to be committed, $b$ the blowup factor, and $N = b \cdot n = 2^k$ the size of the evaluation domain. Two elements are fixed:

$$\omega = \gamma^{(p-1)/2^k}, \qquad g = \gamma^{(p-1)/2^{k+1}}, \tag{2.1}$$

where $\gamma$ is a fixed multiplicative generator of $\mathbb{F}_p^*$. Here $\omega$ is a primitive $2^k$-th root of unity: its powers $\langle\omega\rangle = \{1, \omega, \ldots, \omega^{N-1}\}$ form a cyclic group of $N$ equally spaced points. The element $g$ is a primitive $2^{k+1}$-th root of unity, one "half-step" finer, and

it plays the role of an *offset* that rotates the whole group. The two are linked by the invariant

$$g^2 = \omega, \tag{2.2}$$

which, as we show below, is exactly what makes the domain reproduce itself under squaring.

The evaluation domain is the coset

$$D = g \cdot \langle \omega \rangle = \{\, g\omega^j : j = 0, \ldots, N-1 \,\}, \tag{2.3}$$

i.e. the group $\langle \omega \rangle$ shifted by the offset $g$. The codeword committed by the prover is $\big(f(g\omega^j)\big)_{j=0}^{N-1}$, computed by scaling the $i$-th coefficient of $f$ by $g^i$ and applying the NTT (sections 1.2.1 and 1.2.2). In our implementation $\omega$ and $g$ are returned by `two_adic_generator(k)` and `two_adic_generator(k+1)` respectively, and the scaled NTT is performed by `coset_lde`.

Three properties follow, and together they justify the folding step.

**Proposition 2.1.** *With $D$ as in eq. (2.3) and $N = 2^k$, $k \geq 1$:*

1. *$\omega^{N/2} = -1$;*
2. *$D$ is closed under negation: $-g\omega^j = g\omega^{j+N/2}$ for $j < N/2$;*
3. *$D^{(2)} = \omega \cdot \langle \omega^2 \rangle$, a coset of the subgroup of order $N/2$; in particular $|D^{(2)}| = N/2$.*

*Proof. (1)* The element $\omega^{N/2}$ squares to $\omega^N = 1$, so it is a square root of unity. It cannot equal 1, since $\omega$ has order exactly $N$; as $\mathbb{F}_p$ is a field, the only square roots of unity are $\pm 1$, hence $\omega^{N/2} = -1$.

*(2)* Using (1), $g\omega^{j+N/2} = g\omega^j \cdot \omega^{N/2} = -g\omega^j$, and the index $j+N/2$ lies in $\{0, \ldots, N-1\}$, so the negated element is again in $D$.

*(3)* Squaring an element of $D$ and applying eq. (2.2) gives $(g\omega^j)^2 = g^2\omega^{2j} = \omega^{2j+1}$, so $D^{(2)} = \{\omega^{2j+1} : j = 0, \ldots, N-1\}$, the subgroup $\langle \omega^2 \rangle$ shifted by $\omega$. By (2) the map $x \mapsto x^2$ identifies $x$ with $-x$, so it is two-to-one on $D$ and $|D^{(2)}| = N/2$. □

Property (3) is what makes the construction *self-reproducing*: the squared domain is again a coset of a two-power subgroup, of the same shape as $D$ but half the size, now described by the pair $(\omega, \omega^2)$ in place of $(g, \omega)$. A single FRI round therefore updates the domain simply by squaring both parameters,

$$g \leftarrow g^2, \qquad \omega \leftarrow \omega^2, \tag{2.4}$$

which in our implementation is the update `offset = offset*offset`, `omega = omega*omega` applied at the end of each layer. The requirements of section 2.2.1 are preserved at every level, as long as the subgroup order remains even.

**Example 2.1.** Take $\mathbb{F}_{17}$, whose multiplicative group has order $16 = 2^4$, with generator $\gamma = 3$. For $N = 8$ we get $\omega = 3^2 = 9$ and $g = 3^1 = 3$, so $g^2 = 9 = \omega$ as required by eq. (2.2). The evaluation domain is

$$D = \{3, 10, 5, 11, 14, 7, 12, 6\}.$$

Its elements pair up as $(3, 14)$, $(10, 7)$, $(5, 12)$, $(11, 6)$, each pair summing to 0 mod 17, this is $-g\omega^j = g\omega^{j+4}$ in action. Squaring gives $D^{(2)} = \{9, 15, 8, 2\}$, of size 4, which is exactly the coset $\omega \cdot \langle \omega^2 \rangle = 9 \cdot \langle 13 \rangle$ built from the updated parameters.

## 2.3 The Folding Operation

Folding is the algebraic core of FRI. In one step it takes a polynomial $f$ of degree less than $d$, given by its values on the domain $D$, and produces a new polynomial of degree less than $d/2$, given by its values on the smaller domain $D^{(2)}$. The reduction uses a single random challenge and no interpolation: the values of the new polynomial are a simple combination of the values of the old one. Repeating this step reduces the degree by half each time; the prover stops once the polynomial is small enough to send directly, and transmits its remaining coefficients in the clear.

The section is organised around the three questions that folding answers: how a polynomial splits into two halves (section 2.3.1), how those halves are recovered from evaluations alone (section 2.3.2), and how the challenge combines them into the folded polynomial (section 2.3.3). The two polynomials $f_e$ and $f_o$ introduced next appear throughout the section in three guises. They are first defined as polynomials, then evaluated at a point, then recombined, but they are always the same two objects.

### 2.3.1 Even-Odd Decomposition

The first step is to split $f$ into the part carried by its even-indexed coefficients and the part carried by its odd-indexed ones. Grouping the coefficients this way lets us write $f$ in terms of two polynomials in $X^2$, each of roughly half the degree.

**Definition 2.2** (Even-Odd Decomposition)**.** Let $f(X) = \sum_{i=0}^{d-1} c_i X^i \in \mathbb{F}_p[X]$. Define

$$f_e(Y) = \sum_{i \text{ even}} c_i \, Y^{i/2}, \qquad f_o(Y) = \sum_{i \text{ odd}} c_i \, Y^{(i-1)/2}.$$

Then $f$ decomposes uniquely as

$$f(X) = f_e(X^2) + X \cdot f_o(X^2), \tag{2.5}$$

and both $f_e$ and $f_o$ have degree less than $\lceil d/2 \rceil$.

In words: $f_e$ collects the even coefficients $c_0, c_2, c_4, \ldots$ and $f_o$ the odd ones $c_1, c_3, c_5, \ldots$, each repackaged as a polynomial in $Y = X^2$. Equation (2.5) just puts them back together, with the extra factor $X$ accounting for the odd powers. Each half keeps at most $d/2$ of the original $d$ coefficients, which is why its degree is halved. This is the same split that drives the recursive step of the NTT (section 1.2.1), and it is the decomposition FRI uses in the commit phase.

### 2.3.2 Recovering the Components from a Pair

There is a catch: the prover does not hold $f$ as a list of coefficients, but as a list of values on $D$. So the values of $f_e$ and $f_o$ must be recovered from the values of $f$ alone. The trick is to evaluate $f$ at a point $x$ and at its negative $-x$. Because $(-x)^2 = x^2$, both evaluations land on the same point $x^2$ of the squared domain, and the odd part flips sign:

$$\begin{aligned} f(x) &= f_e(x^2) + x \cdot f_o(x^2), \\ f(-x) &= f_e(x^2) - x \cdot f_o(x^2). \end{aligned} \tag{2.6}$$

These are two linear equations in the two unknowns $f_e(x^2)$ and $f_o(x^2)$. Adding them isolates the even part; subtracting them isolates the odd part. Since $x \neq 0$ for every $x \in D$, we can divide and obtain

$$f_e(x^2) = \frac{f(x) + f(-x)}{2}, \qquad f_o(x^2) = \frac{f(x) - f(-x)}{2x}. \tag{2.7}$$

So a single pair of values $\big(f(x), f(-x)\big)$ determines both components at the point $x^2 \in D^{(2)}$. By theorem 2.1 that pair is stored in the codeword at indices $i$ and $i + N/2$, so the prover finds it by reading two entries that sit exactly half the list apart.

### 2.3.3 The Fold

The final step mixes the two halves using the verifier's random challenge $\alpha$. The mix collapses $f_e$ and $f_o$ into one polynomial of half the degree.

**Definition 2.3** (FRI Fold)**.** Let $f$ have degree less than $d$ and let $\alpha$ be a challenge sampled from a field $\mathbb{K} \supseteq \mathbb{F}_p$. The *fold* of $f$ with respect to $\alpha$ is

$$f^{(\alpha)}(Y) := f_e(Y) + \alpha \cdot f_o(Y), \tag{2.8}$$

a polynomial over $\mathbb{K}$ of degree less than $\lceil d/2 \rceil$.

Putting eq. (2.8) together with eq. (2.7), the values of $f^{(\alpha)}$ on $D^{(2)}$ come straight from the values of $f$ on $D$, with no interpolation anywhere:

$$f^{(\alpha)}(x^2) = \frac{f(x) + f(-x)}{2} + \alpha \cdot \frac{f(x) - f(-x)}{2x}. \tag{2.9}$$

This is exactly what our implementation does: for each $i < N/2$ it reads the two entries at indices $i$ and $i + N/2$, applies eq. (2.9) with $x = g\omega^i$, and writes out a codeword of length $N/2$ on the domain $D^{(2)}$ given by the updated parameters of eq. (2.4).

One round thus halves two things at once: the degree bound, by definition 2.3, and the domain size, by theorem 2.1. The rate $\rho$ stays the same, so the folded codeword is again a Reed-Solomon codeword of the same rate, which is what lets the next round repeat the identical step. The prover keeps folding until the codeword reaches a fixed target length, at which point it sends the remaining layer directly; in our implementation this target is the parameter `final_poly_len`, and the commit phase folds while the codeword is longer than `final_poly_len`.

*Remark* 2.2 (Arithmetic in the extension field). The initial codeword has entries in $\mathbb{F}_p$, but the challenge $\alpha$ is sampled from the quartic extension $\mathbb{F}_{p^4}$ (section 1.1.3). Because eq. (2.8) multiplies $f_o$ by $\alpha$, the folded polynomial already has coefficients in $\mathbb{F}_{p^4}$, and so does every layer after it. In our implementation this switch happens at the very first fold: the initial codeword holds base-field elements, every later layer holds extension-field elements, and the Merkle leaves grow accordingly from four to sixteen bytes.

**Example 2.2.** We continue example 2.1. Let $f(X) = 1 + 2X + 3X^2 + 4X^3$ over $\mathbb{F}_{17}$, so $d = 4$, $N = 8$, and $\rho = 1/2$. Evaluating $f$ on $D = \{3, 10, 5, 11, 14, 7, 12, 6\}$ gives the codeword

$$\big(f(x)\big)_{x \in D} = (6, 3, 8, 15, 16, 4, 8, 16).$$

Splitting $f$ as in definition 2.2 gives $f_e(Y) = 1 + 3Y$ and $f_o(Y) = 2 + 4Y$. With the challenge $\alpha = 5$, the fold is

$$f^{(5)}(Y) = (1 + 3Y) + 5\,(2 + 4Y) = 11 + 6Y,$$

of degree $1 < 2$, exactly as definition 2.3 predicts. Applying eq. (2.9) to the four pairs $\big(f(x), f(-x)\big)$ gives the codeword $(14, 16, 8, 6)$ on $D^{(2)} = \{9, 15, 8, 2\}$, which is precisely the evaluation of $11 + 6Y$ on that domain. Both the degree bound and the domain size have halved, and the rate is still $2/4 = 1/2$.

## 2.4 The Protocol

The folding operation of section 2.3 is the engine; the protocol is what wraps it into an interaction between prover and verifier. It has two phases. In the *commit phase* the prover folds the polynomial down to a short final layer, committing to each intermediate codeword with a Merkle tree so that it cannot change its mind later. In the *query phase* the verifier picks a few random positions and asks the prover to reveal the values needed to re-check that each fold was done correctly. If the prover cheated, that is, if the starting codeword was not close to a low-degree polynomial, then at least one of these checks fails with high probability.

Throughout, $D_0 = D$ is the initial domain of size $N$, and $D_r$ is the domain after $r$ folds, of size $N/2^r$.

### 2.4.1 Commit Phase

The prover starts from the coefficients of $f$, extends them to a codeword on $D$ by the low-degree extension (section 1.2.2), and then repeats the same three actions at every round: commit, receive a challenge, fold.

1. **Commit.** Build a Merkle tree over the current codeword and send its root. This binds the prover to that exact codeword: any later opening must be consistent with the root.

2. **Challenge.** Obtain the folding challenge $\alpha_r$ from the verifier (in the non-interactive setting, from the transcript; see section 2.5).

3. **Fold.** Apply eq. (2.9) with $\alpha_r$ to halve the codeword, and square the two domain parameters (eq. (2.4)) to move to $D_{r+1}$.

The loop stops once the codeword has shrunk to the target length $\ell =$ `final_poly_len`, at which point the prover sends the remaining layer directly. The number of rounds is therefore $\log_2(N/\ell)$.

In our implementation these steps are the body of `commit_phase`: the codeword is produced by `coset_lde`, each root is committed with a `MerkleTree` over `sha256` of the entries, the challenge is drawn by `sample_ext_field`, and `fold` performs the reduction. For the running example, with $f$ of length 8 and blowup 2, so $N = 16$ and $\ell = 2$, the loop runs $\log_2(16/2) = 3$ times, producing three Merkle roots and a final layer of two elements.

### 2.4.2 Query Phase

Committing to the codewords is not enough on its own: the verifier must check that consecutive layers are actually related by the fold. It does so by sampling a small number of random positions and, at each one, asking the prover for just the values needed to recompute one fold.

For a single query the verifier picks an index and follows it down through the layers. At round $r$, with domain size $n_r$, the index determines a pair of positions half the list apart,

$$\text{lo} = \text{idx} \bmod (n_r/2), \qquad \text{hi} = \text{lo} + n_r/2,$$

which are exactly the entries $f(x)$ and $f(-x)$ that eq. (2.9) combines. The prover returns both values together with their Merkle authentication paths. The index for the next round is then $\text{idx} \leftarrow \text{lo}$, so one random choice in the first layer determines the whole chain of positions down to the final layer.

This is realised by `query_phase`: the verifier samples `num_queries` indices in the range $[0, N/2)$, and for each one walks the rounds, opening the two Merkle paths per layer. Each query thus costs one pair of openings per round, that is, $\log_2(N/\ell)$ pairs.

### 2.4.3 Verification and the Colinearity Check

The verifier holds only the proof, namely the Merkle roots, the final layer, and the opened values with their paths, and must decide whether to accept. It proceeds in three steps.

First, it **reconstructs the challenges**. Re-reading the roots and the final layer from the transcript in the same order the prover wrote them, it recomputes every $\alpha_r$ and every domain parameter, and checks that the number of rounds matches the expected $\log_2(N/\ell)$.

Second, for each queried position it **checks the Merkle paths**, confirming that the two opened values really sit at their claimed positions under the committed root. A failed path means the prover is opening something it did not commit to.

Third, it performs the **colinearity check**. Using the opened pair $(\text{lo}, \text{hi})$ at round $r$, it recomputes the folded value by the same rule the prover should have used,

$$v = \frac{\text{lo} + \text{hi}}{2} + \alpha_r \cdot \frac{\text{lo} - \text{hi}}{2\,x}, \qquad x = \text{offset}_r \cdot \omega_r^{\text{lo}}, \tag{2.10}$$

and compares $v$ against the value at the corresponding position of the *next* layer, which is an opened entry for intermediate rounds, or the final layer for the last round. If the two disagree, the fold was not performed honestly and the verifier rejects.

The name *colinearity check* comes from the geometry of eq. (2.10). The two points $(x, \text{lo})$ and $(-x, \text{hi})$, together with the folded value at $\alpha_r$, must lie on a single line, since the line through $f(x)$ and $f(-x)$ evaluated at $\alpha_r$ is exactly $f^{(\alpha_r)}(x^2)$. Each query verifies one such local relation per layer; a codeword far from low degree cannot satisfy all of them at once, so a handful of random queries suffices to catch a cheating prover.

In our implementation this is `fri_verify`: it rebuilds the $\alpha_r$ from the transcript, replays the two `asserts` on the round count and final-layer length, re-derives the query indices, and for each query checks the two Merkle paths with `verify_proof` and the colinearity relation of eq. (2.10) against either the next opened value or `final_poly`.

## 2.5 Fiat-Shamir and the Challenger

The protocol of section 2.4 is interactive: at each round the prover commits and the verifier answers with a random challenge $\alpha_r$. To turn it into a proof that can be written down once and checked offline, the interaction is removed with the Fiat-Shamir transformation (section 1.7.3): instead of waiting for the verifier, the prover derives each challenge itself, by hashing everything it has sent so far. The verifier, holding the same transcript, recomputes the identical challenges and so can replay every check.

The component that maintains this transcript and produces the challenges is the *challenger*. This section describes what it must guarantee (section 2.5.1) and how our implementation realises it with a duplex sponge built on Keccak (section 2.5.2).

### 2.5.1 The Transcript and its Ordering

A challenge is safe only if it depends on everything the prover has committed up to that point. If the prover could choose a Merkle root *after* seeing the challenge that will be applied to it, it could search for a root that makes a dishonest fold pass the colinearity check. Binding each challenge to the full prior transcript removes this freedom: by the time $\alpha_r$ is fixed, the root of layer $r$ is already absorbed and can no longer be changed.

Concretely, the challenger alternates two operations. It *absorbs* the prover's messages, updating an internal state, and it *squeezes* that state to produce challenges. During the commit phase the prover absorbs the parameters, then, for each round, absorbs the Merkle root and squeezes the folding challenge $\alpha_r$; after the last fold it absorbs the final layer. The query indices are squeezed once the whole commit transcript is fixed.

The single rule that makes this sound is that the verifier must reproduce the *same operations in the same order*. Our `fri_verify` does exactly this: it absorbs each root and squeezes each $\alpha_r$ in the same sequence as the prover, absorbs the final layer, and only then squeezes the query indices with `sample_indices`. Any mismatch in order would yield different challenges and reject an honest proof.

### 2.5.2 The Duplex Sponge over Keccak

The absorb and squeeze operations must come from a construction where writing to the transcript and reading from it are genuinely distinct. Chiesa and Orrù [CO25] show that the *duplex sponge* is such a construction and that the Fiat-Shamir transformation built on it preserves the soundness of the underlying interactive protocol. A sponge keeps a state split into two parts: a public *rate*, into which data is absorbed and from which output is squeezed, and a hidden *capacity*, never exposed directly, which carries the security of the construction. Absorbing and squeezing are separate operations on the state, so there is no ambiguity between the prover writing a message and the verifier reading a challenge, which is the property a Merkle-Damgård hash such as SHA-256 does not cleanly provide.

Our challenger instantiates this over the Keccak-$f[1600]$ permutation. The state is 1600 bits, viewed as 25 lanes of 64 bits; the first 17 lanes (136 bytes) are the rate and the remaining 8 lanes (64 bytes) are the capacity. Absorbing XORs each byte into the rate and applies the permutation whenever the rate fills; squeezing reads bytes back

out of the rate, permuting again when it is exhausted. The switch from absorbing to squeezing is marked by the standard Keccak padding.

Two kinds of challenge are drawn from the squeezed bytes:

- The folding challenges $\alpha_r$ are elements of the quartic extension $\mathbb{F}_{p^4}$ (remark 2.2). Each is built by squeezing 16 bytes and reading four base-field coefficients from them, one per 32-bit word. This is `sample_ext_field`.
- The query indices are integers in $[0, N/2)$, each obtained by squeezing bytes and reducing modulo the domain size. This is `sample_indices`.

The whole permutation is implemented from scratch in `keccak.py`, so the challenger depends on no external cryptographic library. This follows the design of the reference implementation `spongefish` that accompanies [CO25], and keeps the trusted computing base of the system limited to the Keccak permutation and the SHA-256 used for the Merkle trees (section 1.5).

*Remark* 2.3 (Grinding). A proof-of-work step, or *grinding*, is often added to the transcript: before sampling the query indices the prover must find a nonce whose hash has a prescribed number of leading zeros, which raises a cheating prover's cost by a fixed number of bits. Our implementation reserves a parameter for this (`proof_of_work_bits`) but leaves the step disabled, so it contributes no additional security in the configuration we evaluate; we return to its effect on the security level in **??**.

# Chapter 3

# FRI as a Polynomial Commitment Scheme

## 3.1 From Low-Degree Testing to Evaluation Proofs

## 3.2 The Quotient Technique

## 3.3 Commit, Open, and Verify

## 3.4 Implementation Notes

# Chapter 4

# Conclusions

## 4.1 Summary

## 4.2 Limitations

## 4.3 Future Work

# Bibliography


[Arn+24] Gal Arnon et al. *WHIR: Reed-Solomon Proximity Testing with Super-Fast Verification*. Cryptology ePrint Archive, Paper 2024/1586. https://eprint.iacr.org/2024/1586. 2024.

[BS+18] Eli Ben-Sasson et al. *Scalable, Transparent, and Post-Quantum Secure Computational Integrity*. Cryptology ePrint Archive, Report 2018/046. https://eprint.iacr.org/2018/046. 2018.

[BSCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. "Interactive Oracle Proofs". In: *Theory of Cryptography Conference (TCC)*. IACR ePrint 2016/116. https://eprint.iacr.org/2016/116. 2016.

[CO25] Alessandro Chiesa and Eylon Orrù. *Fiat-Shamir via Sponges*. Cryptology ePrint Archive, Report 2025/536. https://eprint.iacr.org/2025/536. 2025.

[Sta21] StarkWare. *ethSTARK Documentation*. Cryptology ePrint Archive, Report 2021/582. https://eprint.iacr.org/2021/582. 2021.

[Tha22] Justin Thaler. *Proofs, Arguments, and Zero-Knowledge*. https://people.cs.georgetown.edu/jthaler/ProofsArgsAndZK.html. Foundations, Trends in Privacy, and Security, 2022.